



8

11

1320 sp

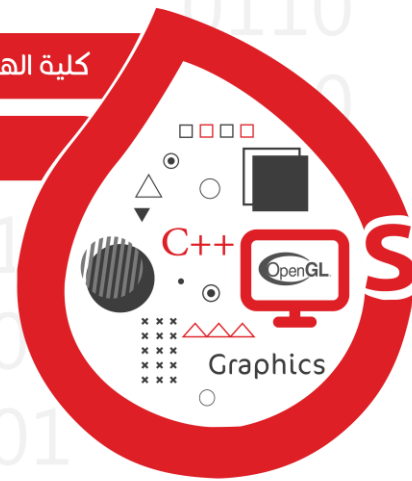
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

scan conversion

د. عمار النحاس



RB Informatics; 25/11/2025

بيانيات حاسوبية

السلام عليكم ورحمة الله وبركاته , تحدثنا في محاضراتنا السابقة عن ال Modeling , Geometrical Transformation, projection, viewing

إن عملية ال Clipping هي اخر خطوة قبل ال projection وهي تتم على ال primitives أي على الخطوط والمضلعات.

سندرس خوارزمية ال clipping لقطعة مستقيمة لأن خوارزميات clipping لل Polygons مشتقة منها.

نستطيع عن طريق هذه الخوارزمية معرفة إذا كانت داخل منطقة العرض أو خارجها أو جزء منها داخل منطقة العرض نتعامل مع قطعة مستقيمة وليس مستقيم لأن القطعة المستقيمة لها نقطة بداية ونقطة نهاية ، اذا الدخل هو نقطة البداية و النهاية للقطعة المستقيمة و الخرج اما داخل او خارج او جزء داخل و جزء خارج منطقة العرض و بالتالي يكون الخرج نقطتين او لا شيء.

■ إن ال Computer Graphics يمر بثلاث مراحل:

1. Modeling: وهي كيفية تمثيل الأجسام، مثال: أعطي الكمبيوتر نقطة بداية ونقطة نهاية.
2. Rendering: وهي كيفية تحويل الأجسام من D3 إلى D2، مثال: بعد إعطاء الكمبيوتر نقطة بداية ونقطة نهاية يقوم بتحويلهم إلى جملة إحداثيات العالم ثم إلى جملة إحداثيات الكاميرا، ثم يطبق خوارزمية Clipping ليتأكد هل سيظهر المستقيم على الشاشة أولاً ثم يطبق Projection فتتحول الإحداثيات من D3 إلى D2، أي إلى جملة إحداثيات الشاشة.
3. Display: من أجل عرض الإحداثيات D2 يجب توافر شاشة عرض مناسبة لتبدأ عملية الرسم الحقيقية

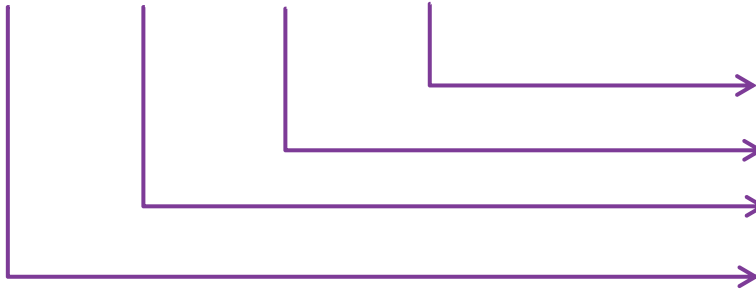
و الان سنتحدث عن الخوارزمية التي تحقق ال clipping على المستقيمت:

خوارزمية Cohen-Sutherland

1001	1000	1010
0001	0000	0010
0101	0100	0110

نقوم بتمديد حواف نافذة العرض لينتج أربعة خطوط نسعي كل منها (up-down-left-right) حسب مكان الخط. ينتج تسعة أقسام نضع لكل قسم bit code-4 ولكل بت من البتات الأربعة دلالة على جهة من جهات نافذة الرؤيا وبذلك فإننا، بقراءة البت نستطيع معرفة مكان القسم الموافق من النافذة.

✦ وتكون دلالات البتات الأربعة كالآتي:



- إذا كانت النقطة يسار منطقة العرض
- إذا كانت النقطة يمين منطقة العرض
- إذا كانت النقطة أسفل منطقة العرض
- إذا كانت النقطة أعلى منطقة العرض

مثال : المنطقة 0101 تقع تحت يسار الشاشة و المنطقة 1001 تقع أعلى يسار الشاشة

ملاحظة اربيسيزية:

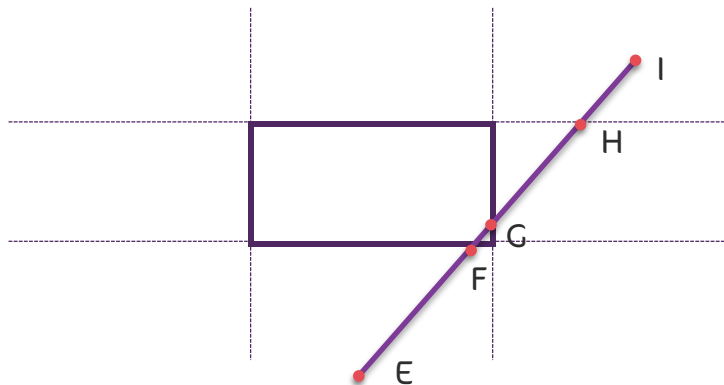
أين تقع النقطة 1100؟

الجواب: مكان غير صالح لأن من المستحيل أن نقول أعلى وأسفل الشاشة

إذاً بداية ونهاية القطعة المستقيمة ستكون في إحدى هذه المناطق، لذا نحسب الرمز بكل من بداية ونهاية القطعة المستقيمة لنعرف إذا كانت تمر من نافذة العرض أم لا:

- إذا كان كلا الرمزین يساوي 0000 فإن القطعة بالكامل داخل منطقة العرض
- إذا لم يكن ناتج عملية (and) بين الرمزین هو 0000 فإن القطعة خارج منطقة العرض وهو شرط لازم لكنه غير كافي أي يمكن أن تقع ويمكن ألا تقع.
- إذا كان ناتج عملية (and) هو 0000 فإن القطعة أو جزء من القطعة المستقيمة داخل منطقة العرض وفي هذه الحالة نقوم بتقسيم القطعة المستقيمة إلى مستقيمين أو أكثر حسب الخطوط التي تتقاطع معها. نأخذ نقطة تقع خارج النافذة أو المنطقة (0000) هذه النقطة هي أحد طرفي القطعة المستقيمة **مثلاً** إذا كانت تقع أعلى النافذة فهي حتماً تتقاطع مع الخط الأعلى up حسب نقطة التقاطع ونهمل القسم الموجود بالكامل خارج النافذة، وتوجد 4-bit code لنقطة التقاطع الجديدة وهكذا حتى نحصل على الجزء المطلوب من القطعة المستقيمة أي طرفا القطعة ضمن المنطقة 0000

مثال:



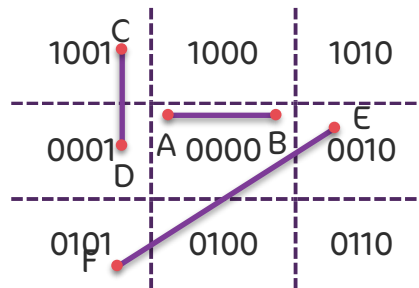
أولاً نقوم بعملية AND بين رمز النقطة I و رمز النقطة E: $AND\ 1010\ 0100 = 0000$ إذاً جزء من القطعة المستقيمة يقع داخل منطقة العرض وجزء خارج منطقة العرض. (الخوارزمية تعتمد على إزالة الواحدات) نبدأ من النقطة E والتي لها الرمز 0100 وحسب هذا الرمز فإن E تقع أسفل منطقة العرض إذاً القطعة EI حتماً تتقاطع مع مستقيم الـ down، ونقطة التقاطع هي F إذاً نطبق الخوارزمية على FI وبما أن رمز F هو 0000، نأخذ النقطة I والتي رمزها 1010 أي تقع فوق ويمين النافذة (لنأخذ أول بت يساوي الواحد) إذاً فالقطعة المستقيمة حتماً تتقاطع مع مستقيم الـ up و نقطة التقاطع هي النقطة H تقع الى يمين منطقة العرض والقطعة تتقاطع مع مستقيم الـ right ونقطة القاطع هي G و رمز النقطة هو 0000 إذاً حصلنا على القطعة FG وهي الجزء الواقع بالكامل داخل منطقة العرض.

ملاحظة: نسمي الحالة التي تكون فيها القطعة المستقيمة بكاملها داخل منطقة العرض Trivial acceptance أي مقبولة بدون شرط نقطة البداية = نقطة النهاية = 0000، ونسمي الحالة التي تكون فيه القطعة المستقيمة غير متقاطعة مع منطقة العرض Trivial rejected أي تكون ناتج عملية الـ and بين نقطة البداية و النهاية لا تساوي الصفر

ملخص شروط التقاطع داخل منطقة العرض:

1. أي قطعة مستقيمة لها ترميز بداية و لها ترميز نهاية
2. أي قطعة ترميز البداية والنهاية لها 0000 فهي تقع داخل منطقة عرض الشاشة
3. إذا أجرينا عملية And بين ترميز البداية والنهاية ولم تكن النتيجة 0000 فلا يوجد أي جزء من القطعة يقع داخل منطقة العرض وإن كانت نتيجة عملية الـ AND هي 0000 فمن الممكن أن يقع داخل المنطقة وممكن ألا يقع

مثال:



لنحل المثال معاً:

- AB تقع داخل منطقة عرض الشاشة لأن ترميز بدايتها ونهايتها 0000 فنسميها Trivial acceptance أي تكون كامل القطعة المستقيمة داخل منطقة العرض
- CD تقع خارج منطقة العرض لأن $D=0001$, $C=1001$ و $D=0001$ و $C \neq 0000$ فنسميها Trivial rejected أي تكون كامل القطعة المستقيمة خارج منطقة العرض

FE جزء القطعة المستقيمة داخل منطقة العرض لأن $F=0101$, $E=0010$ و E and $F=0000$ هنا جزء من القطعة المستقيمة يقع داخل منطقة العرض يجب أن نحدد هذا الجزء ولكن كيف ؟؟

هل يوجد نقطة على يمين منطقة العرض ؟

نعم نوجد نقطة التقاطع بين القطعة المستقيمة و المستقيم right ونستبدل النقطة E بالنقطة E1

هل النقطة E1 فوق منطقة العرض ؟

لا ننتقل للخطوة التالية

هل النقطة E1 تحت منطقة العرض ؟

لا ننتقل للخطوة التالية

بعد المرور على المراحل الأربعة تكون النقطة إما داخل أو على محيط منطقة العرض

نعيد الخطوات على النقطة F :

1. لا: ننقل للخطوة التالية.

2. لا: ننتقل للخطوة التالية.

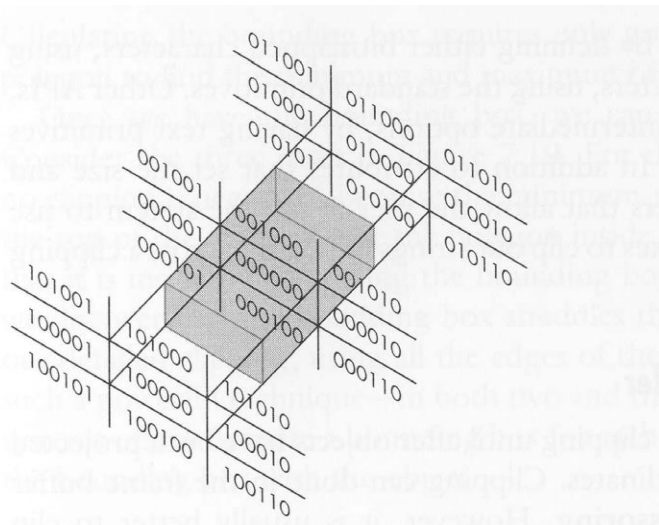
3. نعم: نوجد نقطة التقاطع بين القطعة المستقيمة والمستقيم left ونستبدل النقطة F بنقطة التقاطع F1 .

4. نعم: نوجد نقطة التقاطع بين القطعة المستقيمة والمستقيم down ونستبدل النقطة F1 بنقطة التقاطع F2

5. وتكون القطعة المستقيمة E1F2 واقعة داخل منطقة العرض.

إن هذا يطبق على 2D أما في حالة 3D Clipping:

كل منطقة تحوي 6 bit بدلاً من 4 bit ولدينا 27 منطقة في ثلاثة مستويات أي أصبح لدينا مناطق في الأمام (front) ومناطق في الخلف (behind).

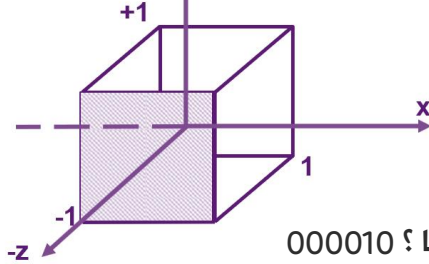


ونميز حالتين حسب نوع الإسقاط:

1. في حالة الاسقاط المتوازي:

شكل منطقة العرض هو متوازي مستطيلات وبما أننا حولنا إلى الشكل القانوني أصبح بإمكاننا حساب موقع النقطة بسهولة، وتكون دلالة كل بت كالآتي:

$$\begin{cases} x=-1, x=1 \\ y=-1, y=1 \\ z=0, z=-1 \end{cases}$$



Bit 1 ← النقطة فوق منطقة العرض $y > 1$

Bit 2 ← النقطة تحت منطقة العرض $y < -1$

Bit 3 ← النقطة يمين منطقة العرض $x > 1$

Bit 4 ← النقطة يسار منطقة العرض $x < -1$

Bit 5 ← النقطة خلف منطقة العرض $z < -1$

Bit 6 ← النقطة أمام منطقة العرض $z > 0$

مثال: النقطة $p(0.2, -0.4, -1.2)$ ما هو الترميز الخاص بها ؟ 000010

2. في حالة الاسقاط المنظوري:

شكل منطقة العرض جذع هرم لذلك المستويات الثلاث لن تكون متساوية، أيضاً بعد التحويل إلى الشكل القانوني يصبح حساب موقع النقاط أسهل.

وتكون دلالة كل بت كالآتي:

Bit 1 ← النقطة فوق منطقة العرض $y > -z$

Bit 2 ← النقطة تحت منطقة العرض $y < z$

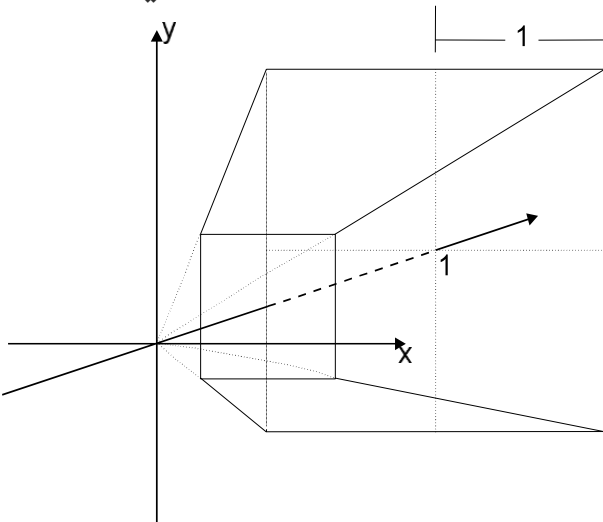
Bit 3 ← النقطة يمين منطقة العرض $x > -z$

Bit 4 ← النقطة يسار منطقة العرض $x < z$

Bit 5 ← النقطة خلف منطقة العرض $z < -1$

Bit 6 ← النقطة أمام منطقة العرض $z > z_{\min}$

مثال: النقطة $p(0.2, -0.4, -1.2)$ ما هو الترميز الخاص بها ؟ 000010



$$x=z, x=-z$$

$$y=z, y=-z$$

$$z=-z_{\min}, z=-1$$

Perspective

سؤال فحص: كم خطوة تحتاج هذه النقطة B لتطبيق clipping على هذه النقطة ؟

نطبق خوارزمية Cohen-Sutherland ونستخرج ترميز النقطة وعدد الواحدات هو ناتج عدد الخطوات

ملاحظة: لا يوجد معادلة مستقيم في 3D بل يوجد حل جملة معادلتين لنستخرج منها معادلة المستقيم

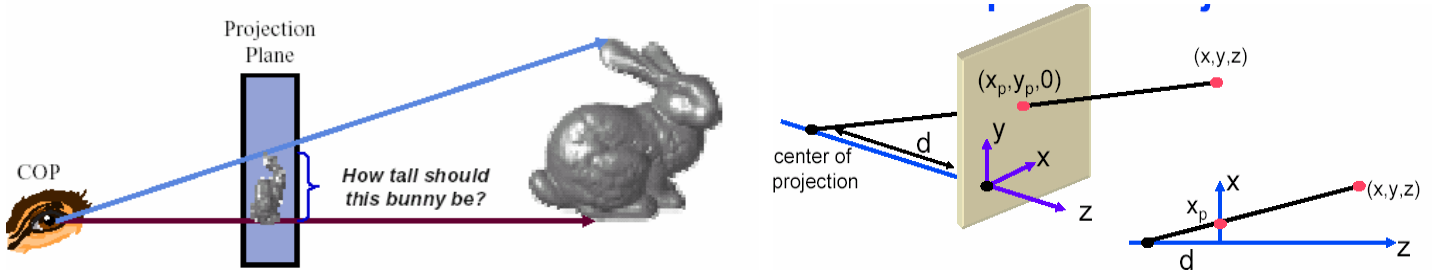
ملاحظة: معادلة المستوي في ال 3D $S: ax + by + cz + d$

معادلة وسيطية للمستقيم و اذا كانت $0 < t < 1$ تكون قطعة مستقيمة

من أجل تقاطع مستقيمين فيكون لدينا t_1, t_2

Perspective Projection

بعد القيام بالعمليات الـ clipping سنحول إلى 2D وذلك يتم عن طريق عمليات الإسقاط:



ستفرض أن:

مستوي الإسقاط هو 2D

الشعاع الناظم VPN هو المحور $z=(0,0,-1)$

Vup هو المحور y (أي جهة للأعلى)

PRP هي $(0,0,-d)$ هي مكان عين المشاهد أو عدسة الكاميرا وهنا نفسها cop

نريد حساب إحداثيات مسقط النقطة (عند أذن الأرنب) على مستوى الإسقاط:

باستخدام تشابه المثلثات:

$$\frac{X_p}{d} = \frac{x}{z+d} \quad ; \quad \frac{Y_p}{d} = \frac{y}{z+d}$$

$$X_p = \frac{d \cdot x}{z+d} \quad ; \quad Y_p = \frac{d \cdot y}{z+d} \quad ; \quad Z_p = 0$$

ملاحظة: نأخذ دوماً z بالقيمة المطلقة $|z|$

$$X_p = \frac{d \cdot x}{z+d} = \frac{x}{z/d + 1} \quad ; \quad Y_p = \frac{d \cdot y}{z+d} = \frac{y}{z/d + 1}$$

حيث d هي المسافة بين مركز الإسقاط cop ومستوي الإسقاط ويحددها المطور

مثال: ما هو مسقط النقطة $(1,2,3)$ حيث $d=1$

نعوض في العلاقات السابقة :

$$x_p = \frac{1 \cdot 1}{3+1} = \frac{1}{4} \quad y_p = \frac{1 \cdot 2}{3+1} = \frac{1}{2}$$

ونعلم ان $Z_p = 0$ إذاً إحداثيات المسقط $(1/4, 1/2, 0)$

إذا كانت قيمة الـ z سالبة اجمع بالقيمة المطلقة

باستخدام المصفوفات و الإحداثيات المتجانسة :

$$M_{per} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{bmatrix} \quad \text{حيث } z = 0, h = \frac{1}{d}$$

H يدل على عملية ال scale التي حدثت بعد الإسقاط

سنقرب المصفوفة بالنقطة للحصول على مسقط هذه النقطة على مستوى الإسقاط:

$$M_{per} P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ \frac{z+d}{d} \end{bmatrix}$$

بعد الضرب علينا القسمة على h حيث $h = \frac{z+d}{d}$ للعودة للإحداثيات العادية .

ملاحظة: تمثل القيمة $h = \frac{z+d}{d}$ مقدار تصغير الجسم (scale) والسبب الإسقاط المنظوري يصغر الأجسام عندما تبعد أما بالنسبة للإسقاط المتوازي لا تتغير إحداثيات x و y لذا سنركز على جعل $z=0$ فتكون المصفوفة :

$$M_{ort} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix}$$

لاحظ هنا أن $h = 1$ حيث أن الإسقاط المتوازي لا يقوم بأي عملية Scale

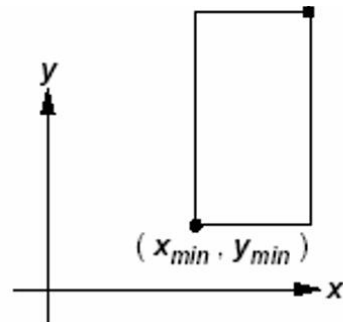
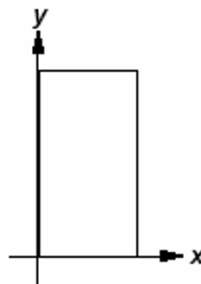
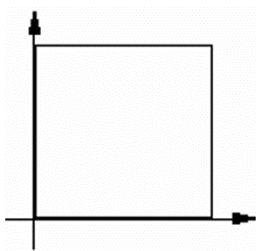
ملاحظة:

إن مصفوفة الإسقاط المتوازي هي نفسها مصفوفة الإسقاط المنظوري ولكن نعلم أنه في الإسقاط المتوازي d تسعى للانهاية:

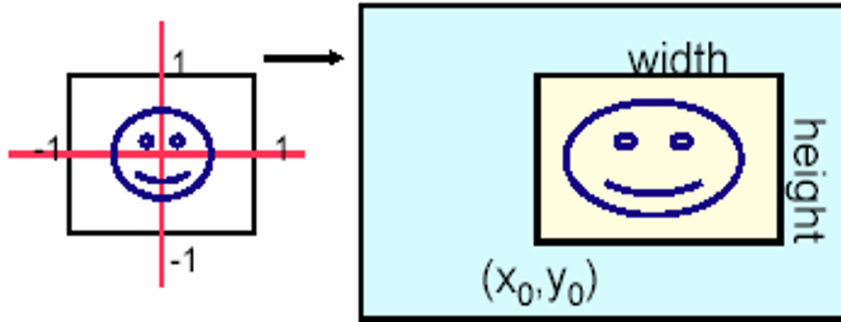
$$\text{When } d=\infty \quad M_{per} = M_{ort}$$

(الانتقال من window إلى viewport) Mapping into a view port:

بعد القيام بعملية الإسقاط حولنا من 3D إلى 2D، لكن نعلم أن الإسقاط لا يغير جملة الإحداثيات أي نحن لازلنا في جملة إحداثيات الكاميرا إذاً علينا القيام بعملية mapping إلى ال view port
يتم ذلك عن طريق translate لل window لتصبح في مبدأ الإحداثيات ثم نقوم بـ scale لتصبح النافذة حجم ال-viewport ثم نقوم بـ translate إلى المكان المطلوب الرسم فيه



هذه التحويلات تدعى viewport transformations و تستخدم لتحويل الاحداثيات ثنائية البعد لتصبح ضمن ال viewport أي من ال word coordinate لل screen coordinate



$$\begin{bmatrix} Xmin \\ Ymin \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{width}{2} & 0 & 0 \\ 0 & \frac{height}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

scale بمقدار $width/2, height/2$ لأننا نعلم ان منطقة عرض ال canonical تساوي 2 من ال -1 الى 1

مثال : (دورة)

View port

$$Up(200,200) \rightarrow (700,400)$$

$$P=(0.5, -0.2)$$

حدد إحداثيات النقطة على الشاشة :

$$Scale = \left(\frac{700-200}{2}, \frac{400-200}{2} \right) \rightarrow Scale = (250, 100)$$

$$Center = \left(\frac{700+200}{2}, \frac{400+200}{2} \right) \rightarrow Center = (450, 300)$$

$$\begin{aligned} \hat{P} &= (250 \times 0.5 + 450, 100 \times (-0.2) + 300) \\ &= (125 + 450, -20 + 300) = (575, 280) \end{aligned}$$

ملاحظة: هل ستتشوه الصورة بنا ان ال scale غير منتظم ؟

لا و ذلك لان نسبة ال Aspect ratio التي وضعناها للكاميرا هي نف هذه النسبة فمنطقة العرض لاصلية لم تكن 1:1 بل نفس النسبة و لكن عندما حولناها الى canonical تشوهت و تعود الى شكلها الطبيعي بعد هذا ال scale

عمليات الإظهار في ال Open تتم في كرت الشاشة مما يكسبها السرعة

لدينا نوعان من الشاشات:

1. Vector Display:

تعتمد على مدفع الإلكترونات في آلية عملها ولا تتعامل مع ال pixels، أي ترسم الخطوط والمنحنيات عن طريق تحريك مدفع الإلكترونات حيث يريد رسم الخط من نقطة البداية إلى نقطة النهاية ومعلومات الرسم هذه تكون مخزنة داخل file أو list

مساوئ ال vector display:

■ في حالة عدد ال vectors الكبير أو الرسم المتحرك (animation) يصبح معدل ال refresh عالي وعمليات مستمرة من التحديث وإعادة الرسم.

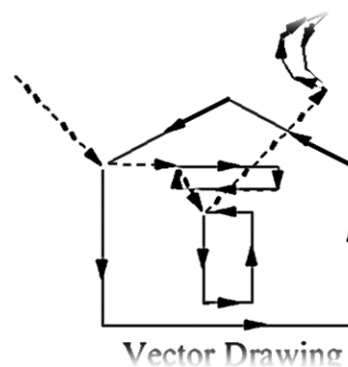
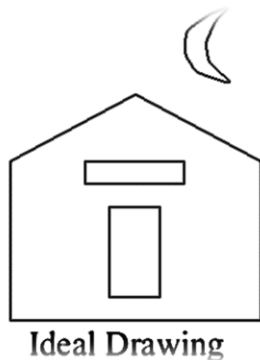
(مثلاً إذا أردنا رسم مثلث فارغ أي غير مظموس نرسم فقط ثلاث خطوط، لكن في حال أردنا مثلث مظموس سيتطلب الكثير من ال vectors إذاً ستبدأ ال vectors التي رسمت بالاختفاء فنحتاج إلى عمليات redrawn refresh.

■ في حالة الرسوم المعقدة يحدث Flicker في الشاشة (تذبذب أو رمش في الشاشة) محدودية ذاكرة تخزين أوامر ومعادلات الرسم (Display list)

■ صعوبة البرمجة

تقنية Vector Display لا تزال تستخدم في طابعات ال potters وفي بعض الأجهزة الطبية، لكنها لم تعد مستخدمة في شاشات العرض الحديثة.

يستخدم مبدأ ال vector في تخزين الصور، حيث يتم تخزين طريقة رسم الصورة في ملف (معادلاتها) بدلاً من تخزين الصورة ك pixels، وتكون مساحة تخزين كملف أقل من مساحة التخزين ك pixels، ولكن هذا فقط في حالة الصورة بعناصر واضحة، فإذا كانت الصورة معقدة يزداد عدد الأوامر في الملف ويصبح حجمه أكبر بكثير من حجم التخزين ك pixels، ويستخدم ال vector في تصميم ال logos أو ال posters حيث تكون العناصر بسيطة وواضحة وبجوانف ناعمة (برنامج illustrator)

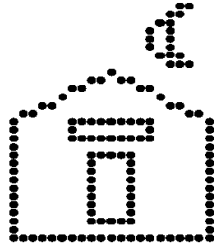


2. Raster Display

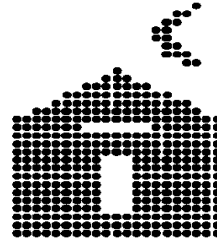
هنا تكون الشاشة مقسمة إلى pixels، لا يهمها في آلية عملها على ما هو الشكل المراد رسمه بل تعتمد على معرفة لون ومكان الـ pixels.

إذاً عملية تحويل أي شكل إلى pixels تسمى Rasterization

تتم عملية الـ Rasterization عن طريق خوارزميات scan conversion وهي خوارزميات لمعادلات الـ primitives لتحويلها إلى pixels، وتكون مخزنة على كرت الشاشة لأنها يجب أن تكون سريعة جداً.



outline primitives



filled primitives

المشكلة في هذا النوع هو التكسر و لكن هناك فائدة اننا استطعنا رسم المنزل ممتلئ و الـ raster يمكننا من رسم صورة معقدة بنفس كلفة الصور البسيطة عن طريق مصفوفة ثنائية البعد من الـ pixels

معلومة هامة للامتحان :

يوجد في كرت الشاشة ذاكرة تسمى Frame buffer يخزن فيها لون كل Pixel في مكان محدد في هذه الذاكرة، و تكون وظيفة كرت الشاشة قراءة الـ frame buffer و تحويلها الى إشارات كهربائية و يرسلها الى الشاشة .

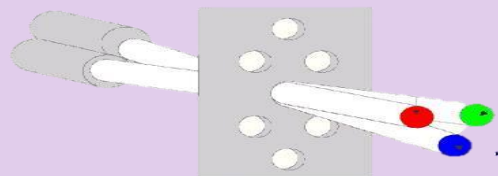
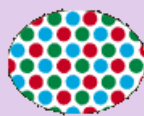
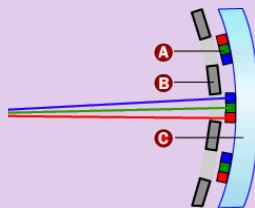
ملاحظة ارييسيزية : في الـ RGB كل واحد منهم يأخذ 1 byte

$$256 \times 256 \times 256 = 16,777,216$$

Output Graphics Devices

تعمل شاشات CRTS عن طريق مدفع الكترونات يطلق الكترونات على اللوحة بلورية مدهونة بمادة فوسفورية تلمع عند اصطدام الالكترون بها، فإذا تم إطلاقه الكترون فان هذا الـ pixel مضيء لونه أبيض، وإذا لم يتم إطلاقه الكترون فإن الـ pixel مطفاً أي لونه أسود، وهنا كانت تستطيع تمييز اللونين الأسود والأبيض فقط. من خلال زيادة عدد الالكترونات المطلقة في المنطقة المضاءة استطعنا الحصول على تدرجات بين الأبيض والرمادي أي كلما كانت الالكترونات المطلقة أكثر اقترب اللون من الأبيض أكثر.

A Phosphors
B Shadow mask
C Glass





Color display : استخدم مبدأ الدمج لالظهار الألوان، حيث تم وضع ثلاث مواد مشعة في كل Pixel، حمراء وخضراء وزرقاء، يتم إطلاقه ثلاث أشعة على كل pixel ونزيد شدة الأشعة عند اللون الذي نريد أن يغلب على ال pixel، ونقوم عين الإنسان بدمج الألوان الثلاث مع بعض، وتم وضع حاجز بين كل بكسلين لتجنب دمج الألوان بين ال pixels المجاورة.

■ كيف يتم عرض الألوان في شاشة CRTS ؟ باستخدام مبدأ الدمج

ملاحظة :

اللون الأصفر الحقيقي الذي نراه في الواقع له طول موجي معين، عندما تصل الموجة. إلى شبكية العين يتم تفسيره على أنه لون أصفر، أما في تقنية الدمج ينتج اللون الأصفر من خلال دمج موجتي اللون الأخضر والأحمر ويتم تفسير هذا الاندماج على أنه لون أصفر

مساوي CRTS:

- تستهلك كهرباء
- تؤدي العينين لأنها تطلق الكترونات و أشعة فوق بنفسجية UV
- وزنها ثقيل

The End...